

Macronutrient Mass Prediction from Food Images

Senén Marqués

June 3, 2026

1 Abstract

2 Aim

The goal of this work is to systematically explore ways in which to predict the macronutrient mass (in grams) from food images.

Predictions were made from 1) A single image and 2) No reference object for size in the image. These rules served to limit the scope of the project, to allow for the use of the same dataset, and to make a general solution that is suitable for real world applications

Obtaining SOTA accuracy presents a significant challenge, and in order to address this, this work adopts both an experimental and a research driven approach. First it is believed that in order to achieve SOTA results, decisions must be anchored in the literature. And second, in order to make a work that is thorough and of decent quality, it should adhere to the scientific method, or at the very least be backed by research and hypothesis testing.

3 The Data

The dataset used is the **MM-Food-100K Dataset**, which contains ~100K labeled images of restaurant and home cooked food.

4 Hardware Limitations

In the last decade, the computer vision landscape has been filled with CNN's, however, in recent years, Vision Transformer have come out and they seem to be the SOTA. While we would like to eventually experiment with Vision Transformers, we are aware that they are computationally expensive and require larger datasets than what we own. Seeing it that

we are working with a **4GB VRAM limitation**, for now we have directed our attention towards the SOTA in CNNs

5 A-Priori Knowledge

A priori, we can tackle the problem with two approaches.

The first is to build a pipeline using various pre-trained models to: 1) Perform segmentation with classification. 2) Predict volumes for each classified segment. 3) Use the the classifications and volumes to trivially obtain the macronutrient mass. We can use models like **FastSAM** for segmentation, **YOLO26** for segmentation with classification, and others for volume estimation. However, the problem with this approach is that any error from a step can compound over to the next step in the pipeline, and if the individual components are not robust or accurate enough, we cannot expect good results.

The second is to simply train a model with our dataset of labeled images and predict macronutrient mass directly. Regardless of the data and CNN architecture used, this is a much simpler and direct way of confronting the problem and the results depend mainly on the dataset and CNN Architecture used.

Naturally, we opted for the second approach as a starting point.

6 Methodology

6.1 Loading the Data

The first step was to download the *MM-Food-100K Dataset*, to do this, we made a python script to download the dataset csv file and images concurrently and idempotently, meaning that upon each

execution, the script will attempt to download any missing images, as it will take several hours to download the whole **166 GB** of images and it is unrealistic to expect the whole script to finish execution uninterrupted in one go.

To ensure maximum speed, our data loading script features concurrency for networking tasks and parallelism for processing and disk access.

After having a local copy of the dataset, we made a custom pytorch Dataset class to allow obtaining the image and targets when iterating over it. This is the Dataset that will be passed to a DataLoader and then to the model.

When testing our custom Dataset class, we noticed warnings due to corrupted images and we modified our script to scan for corrupted files and remove them from the dataset. The script was able to detect some corrupted images but not all, which was frustrating. So, although we were able to use our Dataset for training, we would like improve our data loading script to remove any images that will raise warnings from torchvision’s internal JPEG handling logic.

Later during training, we realized there was a CPU bottleneck caused by the heavy image transform computations from the DataLoader loading batches into the model. We again modified our data loading script to do the heaviest transform, image Resizing, locally. Thus avoiding the CPU bottleneck, and allowing batches to be fed into the model at twice the speed.

We are very eccentric about our code, not only do we want it to be simple and pythonic, we also want it to be performant and reliable. We certainly don’t want to see any warning or errors when loading images as we could very well be **feeding bad data to our model**. On a more vapid tone, we don’t want our terminal to be polluted with warnings every time we run a script or train a model as it looks shabby. For this very reason, we wrote some tests for our data loading script, and will continue revisiting it throughout our work to improve it.

6.2 Choosing a CNN Architecture

We have a computer vision task which requires multi output regression, sadly, we noticed that the litera-

ture on CNNs is very classification biased. We are aware that we can adapt any CNN for regression by modifying the head of the network and changing the criterion from Cross-Entropy loss to MSE (L2) or MAE (L1) loss. However, almost all the CNNs we reviewed in the literature were benchmarked on ImageNet classification and we don’t know which is best suited for multi-output regression. We need to investigate more [1], but as a starting place, we decided to 1) train from scratch and 2) fine tune an existing model.

The most notable one is **EfficientNet** [2], which outperforms many other CNNs in accuracy and computation efficiency by degrees of magnitude. Notably, its microarchitecture features modules made from an MBConv (mobile inverted bottleneck) [3] followed by an SE Block (Squeeze and Excitation Block) [4] and the MBConv features skip connections, all of which are innovations we have encountered in the literature. Its macroarchitecture repeats these modules with different kernel sizes and number of channels.

EfficientNet’s baseline model (b0) was found using *multi objective neural architecture search* that optimises accuracy and FLOPS [2], consequent variants (b1 through b7) were obtained by applying a custom compound scaling method that finds the best constant values for depth, width and resolution and then scales up by a factor of ϕ , efficiently scaling the baseline. We would have liked to train a larger variant of this network but we found ourselves limited by hardware once again.

6.3 Training

We trained EfficientNet_B0 from scratch, after monkey-patching it to suit our multi-output regression needs. We trained it for a period of 50 epochs and obtained an L1 (MAE) loss of 0.15, which we multiplied by each output’s standard deviation to de-standardize it, and found that our 0.15 loss meant an average error of 2.25 grams of fat, 4.35 grams of carbs and 2.55 grams of protein, which gave us a mean error of ~48 Kcal.

We are currently training it for 100 epochs, and will soon obtain its validation error and plot the losses. We are then interested in experimenting with the following:

- Training other models (MobileNetV3)
 - Attempting fine tuning
 - performing data agumentation
 - Replicating the training configurations used in the EfficientNet paper [2]
- [1] S. Kornblith, J. Shlens, and Q. V. Le, “Do better ImageNet models transfer better?” arXiv, 2018. doi: 10.48550/ARXIV.1805.08974.
- [2] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” 2019, doi: 10.48550/ARXIV.1905.11946.
- [3] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” 2018, doi: 10.48550/ARXIV.1801.04381.
- [4] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, “Squeeze-and-excitation networks.” arXiv, 2017. doi: 10.48550/ARXIV.1709.01507.